Python Programming

Complete Reference Notes

From Fundamentals to Modern Python and the Standard Library

1. Introduction to Python

1.1 What is Python?

Python: a high-level, interpreted, general-purpose programming language created by Guido van Rossum, first released in 1991, with a design philosophy that emphasizes code readability and simplicity

Python is dynamically typed and garbage-collected, and supports multiple programming paradigms: procedural, object-oriented, and functional. Its clean, English-like syntax and enormous standard library and third-party ecosystem (PyPI) have made it one of the most widely used languages in the world, spanning web development, data science, automation/scripting, AI/machine learning, and backend systems.

1.2 Key Characteristics of Python

- Interpreted: source code (.py) is executed line-by-line by the Python interpreter (CPython by default), rather than compiled ahead of time into a standalone native executable.
- Dynamically typed: a variable's type is determined at runtime based on the value assigned to it, and can change as the program runs.
- Multi-paradigm: supports procedural, object-oriented, and functional programming styles in one language.
- Indentation-based syntax: code blocks are defined by consistent indentation rather than braces {} — whitespace is syntactically significant.
- Automatic memory management: uses reference counting plus a cyclic garbage collector; there is no manual malloc/free or new/delete.
- Batteries included: a very large standard library ships with every installation, covering everything from file I/O to networking to JSON parsing.
- Huge third-party ecosystem: the Python Package Index (PyPI) hosts hundreds of thousands of installable packages (NumPy, Django, Flask, PyTorch, requests, etc.).

1.3 Python vs C/C++ — Quick Comparison

Aspect	C / C++	Python
Execution	Compiled to native machine code	Interpreted (compiled to bytecode, then run on a virtual machine)
Typing	Static — types fixed at compile time	Dynamic — types determined at runtime
Memory management	Manual (malloc/free, new/delete) or smart pointers	Automatic (reference counting + garbage collector)
Blocks	Curly braces {}	Indentation (whitespace)
Variable declaration	Type must be declared (int x;)	No declaration — just assign (x = 5)

Aspect	C / C++	Python
Performance	Very fast, close to hardware	Slower at runtime; often calls out to C-optimized libraries
File extension	.c / .cpp	.py

1.4 Structure of a Python Program

Python programs have no mandatory main() entry point and no header/include system like C/C++ — execution simply starts at the top of the file and runs sequentially, with function and class definitions creating names as they're encountered.

```
# greet.py
"""A minimal Python program demonstrating basic structure."""

MAX = 100                   # module-level constant (by convention, ALL_CAPS)

def greet(name):              # function definition
    """Print a greeting for the given name."""
    print(f"Hello, {name}!")   # f-string — formatted string literal

def main():
    local_var = 5
    greet("World")
    print("Welcome to Python programming.")

if __name__ == "__main__":     # only runs when this file is executed directly
    main()
```

Explanation of each part

***"""..."""*:** a triple-quoted string; when placed as the first statement of a module, function, or class, it becomes a docstring — a piece of built-in, introspectable documentation

print(): the built-in function used for standard output, roughly analogous to C's printf or C++'s cout

if __name__ == "__main__": a guard that checks whether the file is being run directly (in which case __name__ equals "__main__") versus imported as a module by another file — code inside this block only runs in the former case

> **Note:** Unlike C/C++, Python has no separate compile step you invoke yourself for ordinary scripts — running `python file.py` parses, compiles to bytecode in memory (and often caches it in a __pycache__ folder), and executes it in one step.

1.5 Running a Python Program

```
python3 program.py          # run a script with the Python 3 interpreter
python3 -m module_name      # run an installed module as a script
python3                     # launch the interactive REPL (Read-Eval-Print Loop)
```

Python has evolved through two major incompatible lines: Python 2 (end-of-life since January 2020) and Python 3, which is the only version used in modern development. Within Python 3, minor versions (3.8, 3.9, ..., 3.12+) each add new syntax and standard-library features; this document assumes a recent Python 3 version.

Note: Always use `python3` and `pip3` explicitly on systems where `python`/`pip` might still point to a legacy Python 2 installation. Modern installations increasingly make `python` mean Python 3, but being explicit avoids ambiguity.

2. Variables, Data Types, and Constants

2.1 Variables

Variable: a name bound to an object in memory; unlike C/C++, a Python variable does not have a fixed type — it is simply a label that can be re-bound to objects of any type

```
age = 21            # a name bound to an int object
age = "twenty-one"    # perfectly legal — age now refers to a str object instead
x = y = z = 0         # multiple assignment — all three names bound to 0
a, b = 1, 2           # tuple unpacking — a = 1, b = 2
a, b = b, a           # classic Pythonic swap — no temporary variable needed
```

Note: Because a variable is just a name bound to an object, assignment (`=`) never copies data for mutable objects — it makes a second name refer to the SAME object. `b = a` on a list means `a` and `b` refer to one shared list; mutating through either name is visible through the other.

2.2 Data Types

2.2.1 Core Built-in Types

Type	Example	Description
int	42, -7, 0	Whole numbers of arbitrary precision (no fixed byte size or overflow)
float	3.14, -0.5	Double-precision floating-point number
complex	2+3j	Complex number with real and imaginary parts
bool	True, False	Boolean value; a subtype of int (True == 1, False == 0)
str	"hello"	Immutable sequence of Unicode characters
list	[1, 2, 3]	Mutable, ordered sequence of any objects
tuple	(1, 2, 3)	Immutable, ordered sequence of any objects
dict	{"a": 1}	Mutable mapping of hashable keys to values
set	{1, 2, 3}	Mutable collection of unique, unordered, hashable elements
NoneType	None	Represents the absence of a value (like null/nullptr)

Note: Use the built-in `type()` function to check an object's type at runtime, e.g. `type(age)`, and `isinstance(age, int)` to test it against a type (or tuple of types) — `isinstance` is generally preferred since it also respects inheritance.

2.2.2 Arbitrary-Precision Integers

Unlike C/C++'s fixed-width int (typically 4 bytes, wrapping or overflowing at its limits), Python's int has no fixed size — it automatically grows to represent arbitrarily large numbers, limited only by available memory.

```
big = 2 ** 100
print(big)   # 1267650600228229401496703205376 — computed exactly, no overflow
```

2.2.3 Mutable vs Immutable Types

Mutable object: an object whose internal state CAN be changed after creation without changing its identity — e.g. list, dict, set

Immutable object: an object whose state CANNOT be changed after creation — any 'modification' actually creates a new object — e.g. int, float, str, tuple, bool

```
s = "hello"
s += " world"        # does NOT modify the original string — creates a brand new str object

lst = [1, 2, 3]
lst.append(4)        # DOES modify the list object in place — no new object created
```

2.3 Constants

Python has no true compile-time constant keyword like C++'s const or constexpr. By convention, module-level names written in ALL_CAPS are treated as constants that should not be reassigned, but the language does not enforce this — it is a documentation convention, not a compiler-checked guarantee.

```
PI = 3.14159          # convention: ALL_CAPS signals "treat this as a constant"
MAX_CONNECTIONS = 100
# PI = 3.14           # legal at the language level, but violates the convention
```

Note: For values that genuinely must not change, some codebases use `typing.Final` (from the `typing` module) as a type-checker hint: `MAX: Final = 100`. Like ALL_CAPS, this is only enforced by static type checkers (e.g. mypy), not by the interpreter itself.

2.4 Type Conversion

Implicit type conversion: automatic conversion performed by Python in mixed-type arithmetic, e.g. int automatically becoming float when combined with a float

Explicit type conversion (casting): manually converting a value from one type to another using built-in conversion functions

Function	Purpose
int(x)	Converts x to an integer (truncates floats, parses numeric strings)
float(x)	Converts x to a floating-point number
str(x)	Converts x to its string representation

Function	Purpose
bool(x)	Converts x to True or False using Python's 'truthiness' rules
list(x) / tuple(x) / set(x)	Converts an iterable into the corresponding collection type

```
a = 7
b = 2
result = a / b         # 3.5 — "/" ALWAYS produces a float (true division)
result2 = a // b       # 3   — "//" is floor (integer) division
x = int("42")          # 42 — parses a numeric string
y = float("3.14")      # 3.14
```

Note: Unlike C/C++, Python's single `/` operator always performs "true division" and returns a float, even for two integers (5 / 2 == 2.5). Use the separate `//` (floor division) operator when integer-style division is intended.

2.5 Input and Output

Function	Purpose
print(...)	Writes values to standard output, automatically separated by spaces and ending with a newline
input(prompt)	Reads a line of text from standard input as a string, optionally displaying a prompt

```
name = input("Enter your name: ")     # input() ALWAYS returns a str
age = int(input("Enter your age: "))  # must explicitly convert for numeric use
print("Hello", name, "you are", age, "years old.")
print(f"Hello {name}, you are {age} years old.")   # f-string — cleaner formatting
```

Note: `input()` always returns a `str`, regardless of what the user types — unlike C++'s `cin >> age`, which parses directly into a typed variable. Numeric input must be explicitly converted with `int()` or `float()`, and this conversion raises a `ValueError` if the text isn't a valid number.

2.6 Modules and the import System (preview)

Module: a single .py file (or built-in component) containing Python definitions and statements, which can be imported and reused — Python's rough equivalent of a namespace, covered fully in Section 11

```
import math
print(math.sqrt(16))         # 4.0 — dotted access, similar in spirit to C++'s ::

from math import sqrt, pi
print(sqrt(16), pi)          # imported names used directly, no prefix needed
```

3. Operators and Expressions

3.1 Arithmetic, Comparison, and Logical Operators

Category	Operators
Arithmetic	+ - * / // % **
Comparison	== != > < >= <=
Logical	and or not (short-circuit evaluated, spelled as words rather than &&/||/!)
Bitwise	& | ^ ~ << >>
Assignment	= += -= *= /= //= %= **= &= |= ^= <<= >>=
Identity	is is not (compares OBJECT IDENTITY, not value)
Membership	in not in (tests membership in a sequence/collection)

Note: Python has no ++ or -- increment/decrement operators at all (unlike C/C++) — use `x += 1` / `x -= 1` instead.

3.2 The ** (Exponentiation) Operator

```
print(2 ** 10)     # 1024 — no C/C++ equivalent operator; C++ needs pow() or repeated multiplication
print(9 ** 0.5)    # 3.0  — fractional exponents work directly for roots
```

3.3 == vs is

== *(equality):* compares whether two objects have the SAME VALUE

is (identity): compares whether two names refer to the EXACT SAME OBJECT in memory — the Python analogue of comparing pointer addresses in C/C++

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b)    # True  — same contents
print(a is b)    # False — two distinct list objects with equal contents

c = a
print(a is c)    # True — c is another name for the SAME object as a
```

Note: Use `is` almost exclusively to compare against `None` (`if x is None:`), never to compare general values for equality — small integers and short strings are sometimes cached and "is" by coincidence, but this is an implementation detail that should never be relied upon.

3.4 Chained Comparisons

```
x = 5
print(0 < x < 10)     # True — equivalent to (0 < x) and (x < 10), evaluated once
```

3.5 The Conditional (Ternary) Expression

```
a, b = 10, 20
maximum = a if a > b else b     # equivalent to C++'s (a > b) ? a : b
print(maximum)                  # 20
```

3.6 Membership and Iteration Operators

```
nums = [1, 2, 3, 4]
print(3 in nums)        # True
print(5 not in nums)    # True
print("py" in "python")  # True — substring membership works on strings too
```

3.7 Operator Precedence

Python's precedence rules broadly mirror C/C++ (multiplication/division bind tighter than addition/subtraction, comparisons chain naturally, `and`/`or` behave like &&/||), with `**` binding tighter than unary minus on its left operand — parentheses should be used whenever the intended order isn't immediately obvious.

4. Control Flow Statements

4.1 Indentation Instead of Braces

Python has no {} to delimit blocks. Instead, a colon `:` introduces a block, and CONSISTENT INDENTATION (by convention, 4 spaces) defines which statements belong to it. Mixing tabs and spaces, or inconsistent indentation, causes an IndentationError.

4.2 Decision-Making Statements

4.2.1 if / elif / else

```
marks = 82

if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
elif marks >= 40:
    print("Grade C")
else:
    print("Fail")
```

Note: Python's `elif` is simply a contraction of `else if` — there is no separate `else if` keyword pair as in C/C++.

4.2.2 match Statement (Python 3.10+) — Structural Pattern Matching

match statement: Python's structural pattern-matching construct, introduced in 3.10, offering functionality similar to (but considerably more powerful than) a C/C++ switch statement

```
def describe(grade):
    match grade:
        case "A":
            return "Excellent"
        case "B":
            return "Good"
        case "C" | "D":        # the "|" combines multiple patterns in one case
            return "Passing"
        case _:                # "_" is the wildcard, like switch's default
            return "Invalid grade"
```

Note: Unlike C/C++'s switch, `match` has NO fall-through — there is no need for a `break` statement, and it can match on structure (unpacking tuples, dicts, and class instances), not just simple values.

4.3 Looping (Iteration) Statements

4.3.1 for loop — Iterates Over a Sequence, Not a Counter

for loop: in Python, always iterates directly over the elements of an iterable (list, string, range, etc.) — there is no C-style 'for (init; condition; increment)' form at all

```
marks = [90, 85, 78, 92, 88]
for m in marks:
    print(m, end=" ")
# Output: 90 85 78 92 88

for i in range(1, 6):      # range(start, stop) — stop is EXCLUSIVE
    print(i, end=" ")
# Output: 1 2 3 4 5

for i, m in enumerate(marks):    # enumerate() pairs each element with its index
    print(i, m)
```

Note: `range(1, 6)` behaves like C++'s `for (int i = 1; i < 6; i++)` — the stop value is always EXCLUSIVE. `range(start, stop, step)` also accepts a step, including negative steps to count downward.

4.3.2 while Loop

```
i = 1
while i <= 5:
    print(i, end=" ")
    i += 1
# Output: 1 2 3 4 5
```

Note: Python has no `do-while` loop. The usual workaround is a `while True:` loop with a `break` at the point where the exit condition should be checked, guaranteeing at least one execution.

```
while True:
    response = input("Continue? (y/n): ")
```

```
    if response == "n":
        break
```

4.3.3 The else Clause on Loops — Python-Only

loop else clause: a block attached to a for or while loop that runs ONLY IF the loop completed WITHOUT hitting a break — a construct with no equivalent in C/C++

```
for n in range(2, 10):
    if n % 7 == 0:
        print(f"Found a multiple of 7: {n}")
        break
else:
    print("No multiple of 7 was found.")   # runs only if the loop never broke
```

4.4 Jump Statements

Statement	Effect
break	Immediately terminates the nearest enclosing for/while loop
continue	Skips the rest of the current iteration and moves to the next
pass	A null operation — does nothing; used as a placeholder where a statement is syntactically required
return	Exits the current function, optionally returning a value to the caller

Note: `pass` has no equivalent in C/C++, where an empty block `{}` is enough. Python's colon-and-indent syntax requires SOME statement inside every block, so `pass` fills that role for empty function bodies, empty classes, or stub branches during development.

5. Functions

Function: a named, reusable block of code that performs a specific task, defined in Python with the def keyword

5.1 Function Definition and Call

```
def add(a, b):          # no return type declared — Python infers it at runtime
    return a + b

total = add(3, 4)       # function call
print(total)            # 7
```

Note: Python functions need no forward declaration/prototype the way C/C++ does — a function only needs to be DEFINED before it is CALLED at runtime, in whatever order that happens to occur (typically top-to-bottom in the file, but not always).

5.2 Default Arguments

Default argument: a value automatically used for a parameter when the caller does not supply one — conceptually the same idea as in C++, but Python allows any parameter to have a default regardless of position, subject to ordering rules

```
def greet(name, greeting="Hello"):
    print(f"{greeting}, {name}!")

greet("Ayan")                  # "Hello, Ayan!"
greet("Ayan", "Welcome back")  # "Welcome back, Ayan!"
```

Note: Never use a mutable object (like a list or dict) as a default argument value — default values are evaluated ONCE, when the function is defined, and the SAME object is reused across every call that doesn't override it, leading to a classic and surprising bug: `def f(items=[]):` accumulates items across unrelated calls.

5.3 Keyword Arguments and Argument Order

```
def describe(name, age, city="Unknown"):
    print(f"{name}, {age}, from {city}")

describe("Ayan", 22)                    # positional
describe(age=22, name="Ayan")           # keyword — order doesn't matter
describe("Ayan", age=22, city="Agartala")   # mixed: positional then keyword
```

5.4 *args and **kwargs — Variadic Parameters

args: collects any number of extra POSITIONAL arguments into a tuple inside the function

****kwargs:*** collects any number of extra KEYWORD arguments into a dict inside the function

```
def total(*args):
    return sum(args)
print(total(1, 2, 3, 4))    # 10 — args is the tuple (1, 2, 3, 4)

def show_profile(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")
show_profile(name="Ayan", role="Developer")
```

Note: *args and **kwargs have no direct equivalent in standard C/C++ (C's `printf`-style variadic functions via `<stdarg.h>` are far more restrictive and type-unsafe by comparison). They are the standard Python way to write functions that accept a flexible, arbitrary set of arguments.

5.5 Function Overloading — Not Supported the Same Way

Python has no function overloading in the C++ sense: defining two functions with the same name simply makes the SECOND definition replace the first. The idiomatic alternatives are default arguments, *args/**kwargs, or checking argument types inside a single function.

```
def add(a, b):
    return a + b
def add(a, b, c):    # this REPLACES the previous add — the 2-argument version is gone
    return a + b + c
```

```
# add(2, 3) would now raise a TypeError: missing required argument
```

5.6 Lambda Expressions

Lambda expression: an anonymous, single-expression function defined inline, conceptually the same idea as a C++11 lambda but syntactically simpler and restricted to a single expression (no statements, no multiple lines)

```
square = lambda x: x * x
print(square(5))              # 25

add = lambda a, b: a + b
print(add(3, 4))              # 7

nums = [5, 2, 8, 1]
nums.sort(key=lambda x: -x)     # sort descending using a lambda as the key function
```

Note: Lambdas are most commonly used as short throwaway callbacks passed to functions like `sorted()`, `map()`, `filter()`, and `key=` arguments — for anything longer than one expression, a normal `def` function is clearer and preferred by style guides (PEP 8).

5.7 Recursion

```
def factorial(n):
    if n == 0:          # base case
        return 1
    return n * factorial(n - 1)   # recursive case

# factorial(4) = 4 * 3 * 2 * 1 = 24
```

Note: Python has a default recursion limit (typically 1000 frames, checkable/settable via `sys.getrecursionlimit()` / `sys.setrecursionlimit()`), and unlike some compiled languages, CPython performs NO tail-call optimization — very deep recursion will raise a RecursionError, so an iterative approach is preferred for anything that might recurse deeply.

5.8 Variable Scope: Local, Enclosing, Global, Built-in (LEGB)

LEGB rule: the order Python searches for a name: Local scope first, then any Enclosing function's scope, then the Global (module) scope, and finally Built-in names

```
x = "global"

def outer():
    x = "enclosing"
    def inner():
        x = "local"
        print(x)        # "local" — found in the Local scope first
    inner()

outer()
```

global keyword: used inside a function to declare that an assignment should modify the module-level variable rather than create a new local one

nonlocal keyword: used inside a nested function to declare that an assignment should modify a variable in the nearest ENCLOSING function's scope, not the global scope

```
counter = 0
def increment():
    global counter          # without this, "counter += 1" would raise an error
    counter += 1
```

5.9 Type Hints (preview — expanded in Section 16)

Python remains dynamically typed at runtime, but function signatures can carry optional type hints that document intent and are checked by external tools (mypy, pyright), not by the interpreter itself.

```
def add(a: int, b: int) -> int:
    return a + b
```

6. Strings, Lists, and Tuples

6.1 Strings

str: Python's built-in, IMMUTABLE sequence-of-characters type — there is no separate 'char' type; a single character is just a string of length 1

```
s = "Hello, World!"
print(s[0])            # H — indexing works like an array
print(s[-1])           # ! — negative indices count from the end
print(s[7:12])         # "World" — slicing: [start:stop), stop exclusive
print(s[::-1])         # "!dlroW ,olleH" — reversed, via a step of -1
print(len(s))          # 13
print(s.upper())       # "HELLO, WORLD!"
print(s.replace("World", "Python"))    # "Hello, Python!"
print(s.split(", "))   # ["Hello", "World!"]
```

Note: Every string method (`.upper()`, `.replace()`, etc.) returns a NEW string rather than modifying the original, because str is immutable — this mirrors C++'s std::string in convenience but str can never be mutated in place the way a C char array or a C++ string can via indexed assignment.

6.1.1 String Formatting

Style	Example	Notes
f-strings (3.6+)	f"{name} is {age}"	Preferred modern style — expressions evaluated inline
.format()	"{} is {}".format(name, age)	Older, still widely seen in existing code
% operator	"%s is %d" % (name, age)	Oldest, C-printf-style formatting; largely legacy

```
name, age = "Ayan", 22
print(f"{name} is {age} years old")
```

Style	Example	Notes

```
print(f"{age * 2 = }")          # 3.8+ debug specifier -> "age * 2 = 44"
pi = 3.14159
print(f"{pi:.2f}")              # "3.14" — format spec for 2 decimal places
```

6.1.2 Common String Methods

Method	Purpose
.strip()	Removes leading/trailing whitespace (or given characters)
.split(sep)	Splits a string into a list of substrings
.join(iterable)	Joins an iterable of strings using this string as the separator
.find(sub)	Returns the index of the first occurrence of sub, or -1 if not found
.startswith() / .endswith()	Tests whether the string starts/ends with a given substring
.isdigit() / .isalpha()	Tests whether all characters are digits / alphabetic

```
words = ["Python", "is", "fun"]
sentence = " ".join(words)      # "Python is fun" — note: joined ON the string, not passed like C's strcat
print(sentence.split())         # back to ["Python", "is", "fun"]
```

6.2 Lists

list: a MUTABLE, ordered, resizable sequence that can hold objects of ANY type (even mixed types in the same list) — Python's rough analogue of C++'s std::vector, but heterogeneous and built into the core language

```
marks = [90, 85, 78, 92, 88]
marks.append(95)          # adds to the end — grows automatically, like vector.push_back
marks.pop()               # removes and returns the last element
marks.insert(0, 100)      # inserts 100 at index 0, shifting everything else right
marks.remove(78)          # removes the FIRST occurrence of the value 78
print(marks[0])           # 100
print(len(marks))         # current number of elements
for m in marks:
    print(m, end=" ")
```

Note: As in C arrays, `marks[10]` on a shorter list is out of bounds — but unlike C, Python DOES check this at runtime and raises an IndexError immediately, rather than silently reading adjacent memory as undefined behaviour.

6.2.1 List Slicing

```
nums = [10, 20, 30, 40, 50]
print(nums[1:4])      # [20, 30, 40] — indices 1,2,3
print(nums[:3])       # [10, 20, 30] — from the start
print(nums[2:])       # [30, 40, 50] — to the end
print(nums[::2])      # [10, 30, 50] — every second element
nums[1:3] = [99, 100]  # slice assignment — replaces a whole sub-range in place
```

6.2.2 List Comprehensions

List comprehension: a concise, single-line syntax for building a new list by applying an expression to every element of an iterable, optionally filtered by a condition — no equivalent syntax exists in C/C++

```
squares = [x * x for x in range(1, 6)]            # [1, 4, 9, 16, 25]
evens = [x for x in range(20) if x % 2 == 0]      # filters while building
pairs = [(x, y) for x in range(3) for y in range(2)]  # nested loops in one comprehension
```

Note: List comprehensions are generally preferred over an equivalent `for` loop with repeated `.append()` calls, both for conciseness and for being noticeably faster in CPython due to internal optimizations.

6.3 Tuples

tuple: an IMMUTABLE, ordered sequence — like a list, but once created its contents (and length) can never change

```
point = (3, 4)
print(point[0])       # 3
# point[0] = 10       # TypeError — tuples do not support item assignment

x, y = point          # tuple unpacking
single = (5,)         # a ONE-element tuple needs a trailing comma; (5) is just the int 5
```

Note: Tuples are commonly used for fixed, heterogeneous records (like a C++ std::pair/std::tuple) and as dictionary keys (since they're hashable, whereas lists are not) — choose tuple over list to signal that a sequence's contents are fixed and won't be mutated.

6.4 Copying Lists — Shallow vs Deep

```
import copy
a = [1, 2, [3, 4]]
b = a                 # NOT a copy — b and a are the SAME list object
c = a.copy()          # shallow copy — new outer list, but nested list [3, 4] still shared
d = copy.deepcopy(a)  # deep copy — nested objects are recursively copied too
```

Note: This is one of the most common sources of Python bugs for programmers coming from C/C++: `b = a` never copies a list, dict, or set — it aliases it, similarly to how two pointers can point to the same C array. Use `.copy()`, `list(a)`, or `copy.deepcopy()` when an independent copy is actually needed.

7. Dictionaries and Sets

7.1 Dictionaries

dict: a MUTABLE mapping of unique, hashable keys to values, maintaining insertion order (guaranteed since Python 3.7) — Python's built-in, native equivalent of C++'s std::unordered_map, with no import required

```
ages = {"Ayan": 22, "Riya": 24}
ages["Ayan"] = 23         # update an existing key
ages["Priya"] = 21        # insert a new key
print(ages["Riya"])       # 24
```

```
print(ages.get("Zoya"))      # None — .get() avoids a KeyError for missing keys
print(ages.get("Zoya", 0))   # 0   — with a default value

for name, age in ages.items():
    print(f"{name}: {age}")

del ages["Priya"]            # removes a key-value pair
print('Ayan' in ages)        # True — membership tests against KEYS by default
```

Note: Directly indexing a missing key (`ages["Zoya"]`) raises a KeyError, just as accessing an out-of-range list index raises an IndexError. Use `.get()`, `.setdefault()`, or a `try/except KeyError` when a missing key is a normal, expected possibility rather than a bug.

7.1.1 Dictionary Comprehensions

```
squares = {x: x * x for x in range(1, 6)}    # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

7.2 Sets

set: a MUTABLE collection of unique, unordered, hashable elements — Python's equivalent of C++'s std::unordered_set (with the ordered std::set analogue being achieved via sorted() when needed)

```
s = {5, 1, 3, 1, 5}        # duplicates are automatically discarded
print(s)                   # {1, 3, 5} — order is not guaranteed
s.add(10)
s.discard(1)               # removes 1 if present; no error if absent (unlike .remove())

a = {1, 2, 3}
b = {2, 3, 4}
print(a | b)               # {1, 2, 3, 4} — union
print(a & b)               # {2, 3}       — intersection
print(a - b)               # {1}          — difference
print(a ^ b)               # {1, 4}       — symmetric difference
```

Note: Sets provide average O(1) membership testing, just like dict keys (both are hash-table based internally) — checking `x in some_set` is dramatically faster than `x in some_list` for large collections, where list membership is O(n).

7.3 Container Quick-Reference

Type	Ordered?	Mutable?	Duplicates?	Typical Use Case
list	Yes	Yes	Allowed	General-purpose sequence (default choice)
tuple	Yes	No	Allowed	Fixed records; dictionary keys
dict	Yes (insertion)	Yes	Unique keys	Key-value lookups
set	No	Yes	No (unique)	Membership tests, removing duplicates

Type	Ordered?	Mutable?	Duplicates?	Typical Use Case
frozenset	No	No	No (unique)	An immutable set; usable as a dict key

8. Classes and Objects

Classes are the core of Python's object-oriented model, conceptually similar to C++ classes, but with a simpler, fully dynamic attribute model — attributes can be added to an object at runtime, and there is no separate header/interface file.

8.1 Defining a Class

Class: a user-defined blueprint specifying the attributes and methods every object (instance) of that type will have

Object / Instance: a concrete instantiation of a class, with its own independent set of attribute values

```
class Student:
    def __init__(self, name, roll, marks):    # constructor — called automatically on creation
        self.name = name                       # "self" refers to the instance being created
        self.roll = roll
        self.marks = marks

    def display(self):                         # a method — "self" is always the first parameter
        print(f"{self.name} (Roll: {self.roll}) scored {self.marks}")

s1 = Student("Ayan", 101, 92.5)     # creates an object — __init__ runs automatically
s1.display()                        # Ayan (Roll: 101) scored 92.5
```

Note: `self` is not a keyword — it's simply the conventional name for the first parameter of every instance method, which Python automatically fills in with the object the method was called on (`s1.display()` is really `Student.display(s1)` under the hood). This is analogous to C++'s implicit `this` pointer, but explicit and visible in every method signature.

8.2 Access Control — Convention, Not Enforcement

Python has no public/private/protected keywords. Access levels are signaled entirely by naming convention, and are NOT enforced by the interpreter the way C++'s access specifiers are enforced by the compiler.

Convention	Meaning
name	Public — intended for normal, unrestricted access from anywhere
_name	Protected by convention — 'internal use', but still fully accessible from outside
__name	Name-mangled to _ClassName__name, making accidental external access harder (not impossible)

Convention	Meaning

```
class BankAccount:
    def __init__(self):
        self.__balance = 0.0        # name-mangled — becomes _BankAccount__balance internally

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def get_balance(self):
        return self.__balance
```

Note: This is often summarized as "we're all consenting adults here" — Python trusts the programmer to respect naming conventions rather than the compiler enforcing hard access boundaries as in C++. The double-underscore mangling deters accidental misuse but can still be bypassed deliberately.

8.3 The __init__ Constructor and __del__ Destructor

__init__: the initializer method automatically called immediately after a new instance is created, analogous to a C++ constructor — note that object creation itself happens in a separate, rarely-overridden method called __new__

__del__: a finalizer method called when an object is about to be garbage-collected — analogous to a C++ destructor, but its exact TIMING is not guaranteed (unlike C++'s deterministic destruction), since it depends on CPython's reference-counting and garbage collector

```
class Logger:
    def __init__(self):
        print("Logger created")

    def __del__(self):
        print("Logger destroyed")

log1 = Logger()     # "Logger created"
del log1            # usually triggers "Logger destroyed" immediately (CPython refcounting)
```

Note: Because __del__'s timing isn't guaranteed across all Python implementations, deterministic cleanup (closing files, releasing locks) should use a context manager (`with` statement, Section 10.4) rather than relying on __del__ — this is Python's answer to C++'s RAII.

8.4 Instance Attributes vs Class Attributes

Class attribute: a variable defined directly in the class body, SHARED by all instances unless an instance explicitly overrides it — analogous to a C++ static data member

Instance attribute: a variable set via self.attr inside a method (typically __init__), unique to each individual object

```
class Counter:
    count = 0               # class attribute — shared by all instances

    def __init__(self):
        Counter.count += 1      # modifies the SHARED class attribute
```

```
a, b, c = Counter(), Counter(), Counter()
print(Counter.count)          # 3 — shared across every instance
```

8.5 Instance Methods, Class Methods, and Static Methods

Kind	Decorator	First Parameter	Typical Use
Instance method	(none)	self (the instance)	Operates on one specific object's data
Class method	@classmethod	cls (the class itself)	Alternative constructors; operations on class-level state
Static method	@staticmethod	(none)	A utility function logically grouped with the class, needing neither self nor cls

```
class Pizza:
    def __init__(self, toppings):
        self.toppings = toppings

    @classmethod
    def margherita(cls):                # alternative constructor
        return cls(["mozzarella", "basil"])

    @staticmethod
    def is_valid_topping(name):             # doesn't need self OR cls
        return name.isalpha()

p = Pizza.margherita()
print(Pizza.is_valid_topping("olives"))     # True
```

8.6 Properties — Controlled Attribute Access

@property: a decorator that lets a method be accessed with plain attribute syntax (no parentheses), enabling validation or computed values while keeping the calling code looking like simple attribute access — Python's alternative to explicit getter/setter methods

```
class Circle:
    def __init__(self, radius):
        self._radius = radius

    @property
    def radius(self):
        return self._radius

    @radius.setter
    def radius(self, value):
        if value < 0:
            raise ValueError("radius cannot be negative")
        self._radius = value

    @property
```

```
    def area(self):                  # a computed, READ-ONLY property
        return 3.14159 * self._radius ** 2

c = Circle(5)
print(c.radius, c.area)   # accessed like plain attributes, no ()
c.radius = 10             # goes through the validating setter
```

8.7 Dunder (Magic) Methods and Operator Overloading

Dunder method: a method with a double-underscore name (e.g. __init__, __str__, __add__) that Python calls automatically in response to built-in syntax or functions — the mechanism behind operator overloading and behind constructs like len(obj) or print(obj)

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, other):       # enables the "+" operator for Point objects
        return Point(self.x + other.x, self.y + other.y)

    def __eq__(self, other):        # enables "=="
        return self.x == other.x and self.y == other.y

    def __str__(self):              # called by print() and str()
        return f"({self.x}, {self.y})"

p1, p2 = Point(2, 3), Point(4, 5)
p3 = p1 + p2              # calls __add__
print(p3)                 # (6, 8) — calls __str__
```

Dunder Method	Triggered By
__init__	Object creation: ClassName(...)
__str__ / __repr__	str(obj) / print(obj) vs repr(obj) and the interactive interpreter
__len__	len(obj)
__eq__, __lt__, ...	==, <, and other comparison operators
__add__, __sub__, ...	+, -, and other arithmetic operators
__getitem__	obj[key] — indexing/subscripting
__iter__ / __next__	for x in obj — makes an object iterable
__enter__ / __exit__	with obj: — makes an object a context manager (Section 10.4)

8.8 Structs? — dataclasses (preview, expanded in Section 16)

Python has no separate 'struct' keyword; a plain data-holding class can be written far more concisely using the @dataclass decorator from the standard library, which auto-generates __init__, __repr__, and __eq__.

```
from dataclasses import dataclass

@dataclass
class Point:
    x: int
    y: int

p = Point(2, 3)     # __init__ generated automatically
print(p)            # Point(x=2, y=3) — __repr__ generated automatically
```

9. Inheritance and Polymorphism

9.1 Basic Inheritance

Inheritance: a mechanism letting one class (the subclass/child) acquire the attributes and methods of another (the superclass/parent/base class), modeling an 'is-a' relationship and enabling code reuse

```
class Animal:                       # base class
    def __init__(self, name):
        self.name = name

    def eat(self):
        print(f"{self.name} is eating.")

class Dog(Animal):                  # Dog inherits from Animal
    def __init__(self, name):
        super().__init__(name)      # calls the PARENT's __init__

    def bark(self):
        print(f"{self.name} says Woof!")

d = Dog("Rex")
d.eat()     # inherited from Animal: "Rex is eating."
d.bark()    # Dog's own method: "Rex says Woof!"
```

super(): a built-in function returning a proxy object that lets a subclass call methods (most commonly __init__) on its parent class, without hard-coding the parent's class name — the Python idiom equivalent to C++'s explicit BaseClassName() constructor call

9.2 Multiple Inheritance and the MRO

Unlike Java (and unlike C++'s more restrictive support), Python fully supports multiple inheritance — a class can inherit from several base classes at once. Python resolves attribute/method lookup order using the C3 linearization algorithm, exposed as a class's Method Resolution Order (MRO), viewable via ClassName.__mro__ or ClassName.mro().

```
class Flyable:
    def fly(self):
        print("Flying...")

class Swimmable:
    def swim(self):
        print("Swimming...")

class Duck(Flyable, Swimmable):     # inherits from BOTH
    pass

d = Duck()
d.fly()    # from Flyable
d.swim()   # from Swimmable
print(Duck.__mro__)   # shows the exact lookup order Python uses
```

9.3 Method Overriding

Method overriding: when a subclass defines a method with the same name as one in its base class, replacing the base version for instances of the subclass — this works automatically in Python and requires no special keyword, unlike C++'s virtual/override machinery

```
class Animal:
    def speak(self):
        print("Some generic animal sound")

class Cat(Animal):
    def speak(self):          # overrides Animal.speak
        print("Meow!")

a = Animal()
c = Cat()
a.speak()    # "Some generic animal sound"
c.speak()    # "Meow!"
```

Note: Every method in Python is effectively 'virtual' by default — Python always looks up a method dynamically based on the object's actual runtime type, exactly like a C++ virtual function, with no separate keyword needed. This is a direct consequence of Python's dynamic typing: there is no 'devirtualized' non-virtual method call to opt out of.

9.4 Polymorphism — Duck Typing

Duck typing: Python's characteristic style of polymorphism: "if it walks like a duck and quacks like a duck, it's a duck" — code that calls obj.speak() will happily work on ANY object that has a speak() method, regardless of its class or inheritance hierarchy, with no shared base class or interface required

```
class Cat:
    def speak(self):
        return "Meow!"

class Robot:                # unrelated to Cat, no common base class
    def speak(self):
        return "Beep boop!"

def make_it_speak(thing):    # works with ANY object that has .speak()
    print(thing.speak())

make_it_speak(Cat())
make_it_speak(Robot())
```

Note: This is a meaningfully different default from C++, where runtime polymorphism REQUIRES an explicit shared base class and virtual functions (Section 11.3 of the C++ notes) — Python's duck typing achieves a similar practical effect with no inheritance relationship at all, since method calls are resolved dynamically by name regardless of type.

9.5 Abstract Base Classes

Abstract Base Class (ABC): a class that defines a required interface (via the abc module) but cannot be instantiated directly, forcing subclasses to implement specific methods — Python's equivalent of C++'s abstract classes with pure virtual functions

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):             # no body — must be implemented by subclasses
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):                # MUST implement area(), or Circle stays abstract
        return 3.14159 * self.radius ** 2

# s = Shape()         # TypeError — cannot instantiate an abstract class
c = Circle(5)
print(c.area())       # 78.53975
```

9.6 isinstance() and issubclass()

```
print(isinstance(c, Circle))   # True
print(isinstance(c, Shape))    # True — Circle IS-A Shape through inheritance
print(issubclass(Circle, Shape))  # True
```

10. Exception Handling

Exception handling: Python's structured mechanism for detecting and responding to runtime errors without crashing the program, conceptually identical in spirit to C++'s try/throw/catch but spelled try/raise/except and used far more pervasively in idiomatic Python code

10.1 try, except, else, and finally

```
def divide(a, b):
    try:
        result = a / b
    except ZeroDivisionError as e:      # catches a SPECIFIC
exception type
        print(f"Error: {e}")
        return None
    else:
        print("Division succeeded")      # runs ONLY if no exception
was raised
        return result
    finally:
        print("Division attempt finished")  # ALWAYS runs, error or
not

divide(10, 0)
divide(10, 2)
```

Clause	Purpose
try	Wraps code that might raise an exception
except	Handles an exception matching the specified type (or types)
else	Runs only if the try block completed with NO exception raised
finally	Always runs, whether or not an exception occurred — used for guaranteed cleanup

10.2 raise — Signaling an Exception

```
def withdraw(balance, amount):
    if amount > balance:
        raise ValueError("Insufficient funds for this withdrawal.")
    return balance - amount

try:
    withdraw(100, 500)
except ValueError as e:
    print(e)        # "Insufficient funds for this withdrawal."
```

10.3 Built-in Exception Hierarchy

All built-in exceptions derive from BaseException, with almost all everyday exceptions deriving specifically from Exception.

Exception Class	Typically Raised For
Exception	The common base class for nearly all exceptions a program should catch
ValueError	A function received an argument of the right type but an invalid value
TypeError	An operation was applied to an object of an inappropriate type
KeyError	A dictionary lookup used a key that doesn't exist
IndexError	A sequence was indexed out of range
ZeroDivisionError	Division or modulo by zero
FileNotFoundError	An attempt to open a file that doesn't exist
AttributeError	An attribute or method lookup failed on an object

```
try:
    risky_operation()
except (KeyError, IndexError) as e:      # catching multiple types in one clause
    print(f"Lookup error: {e}")
except Exception as e:                   # catches any other standard exception
    print(f"Other error: {e}")
```

Note: Prefer catching the MOST SPECIFIC exception type that makes sense (`except ValueError` rather than a bare `except:`), and order multiple except clauses from most specific to most general — a bare `except:` (with no type at all) also silently swallows things like KeyboardInterrupt and should generally be avoided.

10.4 Custom Exceptions

```
class InsufficientFundsError(Exception):
    """Raised when a withdrawal exceeds the available balance."""
    pass

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFundsError(f"Cannot withdraw {amount} from {balance}")
    return balance - amount
```

10.5 Context Managers — the with Statement

Context manager: an object implementing __enter__ and __exit__, used with the with statement to guarantee that setup and cleanup code run reliably — Python's structured alternative to C++'s RAII/destructor-based cleanup, and the idiomatic replacement for manual try/finally cleanup

```
with open("data.txt", "r") as f:      # __enter__ opens the file; __exit__ closes it
    contents = f.read()
# the file is GUARANTEED to be closed here, even if an exception occurred above

import time

class Timer:                          # a minimal custom context manager
```

```
    def __enter__(self):
        self.start = time.time()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        elapsed = time.time() - self.start
        print(f"Elapsed: {elapsed:.4f}s")

with Timer():
    sum(range(10**6))
```

Note: `with` is the idiomatic Python replacement for the try/finally cleanup pattern shown in Section 10.1 — it exists specifically because __del__'s garbage-collection timing (Section 8.3) is not guaranteed, so anything requiring deterministic cleanup (files, network sockets, locks) should use a context manager instead of relying on an object's destructor.

11. Modules and Packages

Module: a single .py file containing Python definitions (functions, classes, variables) that can be imported and reused elsewhere — Python's rough equivalent of a C++ translation unit combined with a namespace

Package: a directory containing multiple related modules, marked (in older Python) by an __init__.py file — Python's equivalent of a multi-file library organized into a folder structure

11.1 Importing Modules

```
import math                 # imports the whole module; access via math.sqrt(...)
print(math.sqrt(16))        # 4.0

import math as m            # aliasing — common for long or frequently-used names
print(m.pi)

from math import sqrt, pi   # import specific names directly into the current namespace
print(sqrt(16), pi)

from math import *          # imports EVERYTHING — generally discouraged, see note below
```

Note: `from module import *` is the Python parallel to C++'s discouraged `using namespace std;` at global scope (Section 2.6 of the C++ notes) — it pollutes the current namespace and makes it unclear where a given name came from. Prefer explicit imports (`import math`, or `from math import sqrt`) in real code.

11.2 Writing Your Own Modules

```
# mathutils.py
def square(x):
    return x * x

PI_APPROX = 3.14159

# main.py
import mathutils
```

```
print(mathutils.square(5))        # 25
print(mathutils.PI_APPROX)
```

11.3 Packages and __init__.py

```
myproject/
    __init__.py
    geometry/
        __init__.py
        shapes.py
        transforms.py
    main.py

# main.py
from myproject.geometry import shapes
from myproject.geometry.shapes import Circle
```

Note: Since Python 3.3, __init__.py is technically optional for a directory to be treated as a (namespace) package, but including it remains standard practice for ordinary packages — it can also be used to control what a package exposes and to run package-level setup code.

11.4 The Python Standard Library — Overview

Python ships with an extensive standard library, requiring no separate installation — this is frequently summarized as Python's 'batteries included' philosophy. Selected modules relevant to everyday programming are surveyed in Section 15.

11.5 Installing Third-Party Packages with pip

```
pip install requests           # installs the "requests" package from PyPI
pip install requests==2.31.0    # install a specific version
pip list                       # list installed packages
pip freeze > requirements.txt   # snapshot installed packages and versions to a file
pip install -r requirements.txt # install everything listed in a requirements file
```

Note: pip is covered in depth alongside virtual environments in Section 17, since installing packages safely in real projects almost always involves an isolated virtual environment rather than installing directly into a system-wide Python installation.

12. Iterators, Generators, and Comprehensions

12.1 Iterables and Iterators

Iterable: any object capable of returning its elements one at a time — anything that implements __iter__, including list, tuple, dict, set, str, and range

Iterator: an object that implements BOTH __iter__ (returning itself) and __next__ (returning the next element, or raising StopIteration when exhausted) — the object actually driving a for loop under the hood

```
nums = [10, 20, 30]
it = iter(nums)         # get an iterator FROM the iterable list
print(next(it))         # 10
print(next(it))         # 20
print(next(it))         # 30
# print(next(it))       # StopIteration — the iterator is exhausted
```

Note: A `for x in nums:` loop is really syntactic sugar for repeatedly calling `iter()` then `next()` until StopIteration is raised — this is Python's uniform mechanism for iteration, playing a role broadly analogous to C++'s begin()/end() iterators, but built directly into every for loop rather than requiring an explicit iterator type per container.

12.2 Generator Functions — yield

Generator function: a function containing one or more yield statements, which — instead of running to completion and returning once — produces a sequence of values LAZILY, pausing its execution state between each yield and resuming exactly where it left off on the next call to next()

```
def count_up_to(n):
    i = 1
    while i <= n:
        yield i          # pauses here, returning i, and resumes from this point next call
        i += 1

for num in count_up_to(5):
    print(num, end=" ")
# Output: 1 2 3 4 5

gen = count_up_to(3)
print(next(gen))    # 1
print(next(gen))    # 2 — execution RESUMED from after the yield, not restarted
```

Note: Generators produce values one at a time and on demand, WITHOUT building the entire sequence in memory first — `count_up_to(10**9)` uses almost no memory regardless of n, unlike returning a full list of a billion numbers. This lazy-evaluation model has no direct built-in equivalent in C/C++ (which would require manually implementing an iterator class or coroutine to achieve the same effect).

12.3 Generator Expressions

Generator expression: syntax nearly identical to a list comprehension, but using parentheses instead of square brackets, producing a lazy generator instead of building the full list immediately

```
squares_list = [x * x for x in range(1000000)]    # builds a full list in memory NOW
squares_gen = (x * x for x in range(1000000))     # builds nothing yet — lazy

total = sum(x * x for x in range(1000000))        # memory-efficient — values generated on the fly
```

12.4 Comprehension Summary

Syntax	Produces	Example
[expr for x in it]	list	[x*x for x in range(5)]
{expr for x in it}	set	{x % 3 for x in range(10)}
{k: v for x in it}	dict	{x: x*x for x in range(5)}
(expr for x in it)	generator	(x*x for x in range(5))

12.5 itertools — Building-Block Iterators (preview)

The standard library's itertools module (surveyed further in Section 15) provides a large collection of fast, memory-efficient iterator building blocks — chain(), zip_longest(), permutations(), combinations(), islice(), and more — for composing complex iteration patterns without writing manual loops.

13. Functional Programming Tools

13.1 Functions as First-Class Objects

In Python, functions are objects like any other value: they can be assigned to variables, stored in data structures, and passed as arguments to — or returned from — other functions, without any special syntax like C++'s function pointers or std::function wrapper.

```
def shout(text):
    return text.upper() + "!"

greeting_fn = shout          # no parentheses — this stores the FUNCTION OBJECT itself
print(greeting_fn("hello"))   # "HELLO!"

def apply_twice(fn, value):    # a function accepting another function as a parameter
    return fn(fn(value))

print(apply_twice(shout, "hi"))   # "HI!!"
```

13.2 map(), filter(), and reduce()

Function	Purpose
map(fn, iterable)	Applies fn to every element, returning a lazy iterator of results
filter(fn, iterable)	Keeps only elements for which fn(element) is truthy, lazily
functools.reduce(fn, iterable)	Cumulatively applies a two-argument fn, reducing the iterable to one value

```
nums = [1, 2, 3, 4, 5]

squares = list(map(lambda x: x * x, nums))          # [1, 4, 9, 16, 25]
evens = list(filter(lambda x: x % 2 == 0, nums))      # [2, 4]

from functools import reduce
total = reduce(lambda acc, x: acc + x, nums)          # 15 — cumulative sum
```

Function	Purpose

Note: In idiomatic modern Python, a list/dict/set comprehension (Section 12.4) is usually preferred over map()/filter() for readability — `[x*x for x in nums if x % 2 == 0]` is generally considered clearer than the equivalent chained map()/filter() calls, though both are common enough to recognize in existing code.

13.3 Decorators

Decorator: a function that takes another function (or class) as input and returns a modified/wrapped version of it, applied using the @decorator_name syntax directly above a definition — used to add cross-cutting behaviour (logging, timing, access control, caching) without altering the original function's code

```
import functools
import time

def timer(func):
    @functools.wraps(func)          # preserves the original function's name/docstring
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        print(f"{func.__name__} took {time.time() - start:.4f}s")
        return result
    return wrapper

@timer                              # equivalent to: slow_square = timer(slow_square)
def slow_square(n):
    time.sleep(0.1)
    return n * n

print(slow_square(5))
```

Note: Decorators are the mechanism behind several features already introduced: @staticmethod and @classmethod (Section 8.5) and @property (Section 8.6) are all built-in decorators — @dataclass (Section 8.8) is a class decorator. Writing custom decorators applies exactly the same wrapping principle shown above.

13.4 Closures

Closure: a nested (inner) function that 'remembers' and can access variables from its enclosing function's scope, even after the enclosing function has finished executing — the mechanism decorators rely on internally to keep hold of the wrapped function

```
def make_multiplier(factor):
    def multiply(x):
        return x * factor       # "factor" is captured from the enclosing scope
    return multiply
```

```
double = make_multiplier(2)
triple = make_multiplier(3)
print(double(5), triple(5))       # 10 15 — each closure remembers its OWN "factor"
```

14. File Handling

14.1 Opening and Closing Files

Mode	Meaning
"r"	Read (default) — file must already exist
"w"	Write — creates the file, or TRUNCATES (erases) it if it already exists
"a"	Append — writes are added to the end of the file, preserving existing content
"x"	Exclusive creation — fails if the file already exists
"b"	Binary mode modifier, combined with the above, e.g. "rb", "wb"

14.2 Writing to a File

```
with open("data.txt", "w") as f:      # "with" guarantees the file is closed automatically
    f.write("Name: Ayan\n")
    f.write("Score: 95\n")
# no explicit f.close() needed — the context manager (Section 10.5) handles it
```

Note: The `with` statement is the standard, idiomatic way to work with files in Python — it plays the same role that RAII-based file streams (ofstream/ifstream, Section 16 of the C++ notes) play in C++, guaranteeing the file is closed even if an exception occurs partway through the block.

14.3 Reading from a File

```
with open("data.txt", "r") as f:
    contents = f.read()          # reads the ENTIRE file as one string
print(contents)

with open("data.txt", "r") as f:
    for line in f:               # iterating a file object yields it line by line — memory efficient
        print(line.strip())      # .strip() removes the trailing newline

with open("data.txt", "r") as f:
    lines = f.readlines()        # reads ALL lines into a list of strings at once
```

14.4 Working with CSV Files

```
import csv

with open("students.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["Name", "Score"])
    writer.writerow(["Ayan", 95])

with open("students.csv", "r") as f:
    reader = csv.DictReader(f)        # reads each row as a dict keyed by header
    for row in reader:
        print(row["Name"], row["Score"])
```

14.5 Working with JSON

```
import json

data = {"name": "Ayan", "skills": ["Python", "C++"], "score": 95}

with open("data.json", "w") as f:
    json.dump(data, f, indent=2)      # serialize a dict/list to a JSON file

with open("data.json", "r") as f:
    loaded = json.load(f)             # parse JSON back into Python objects

text = json.dumps(data)               # serialize to a JSON-formatted STRING (not a file)
parsed = json.loads(text)             # parse a JSON string back into Python objects
```

14.6 Working with File Paths — pathlib

pathlib.Path: an object-oriented representation of a filesystem path, introduced in Python 3.4, offering a more readable and cross-platform alternative to string-based path manipulation via os.path

```
from pathlib import Path

p = Path("data") / "students.csv"     # "/" is overloaded to JOIN path components
print(p.exists())                     # True/False
print(p.suffix)                       # ".csv"
print(p.stem)                         # "students"
for file in Path(".").glob("*.py"):       # iterate matching files
    print(file)
```

15. The Python Standard Library — Key Modules

Python's 'batteries included' standard library covers an enormous range of everyday needs without any pip install. A few of the most commonly used modules are surveyed here.

15.1 collections — Specialized Container Types

Type	Purpose

Type	Purpose
Counter	A dict subclass for counting hashable items — Counter(word_list) tallies occurrences
defaultdict	A dict that auto-creates a default value for missing keys instead of raising KeyError
OrderedDict	A dict that remembers insertion order (largely superseded by plain dict since 3.7)
namedtuple	A tuple subclass with named fields, accessible by name as well as by index
deque	A double-ended queue with fast O(1) appends/pops from BOTH ends

```
f
rom collections import Counter, defaultdict, namedtuple, deque

counts = Counter(["a", "b", "a", "c", "a"])
print(counts)                    # Counter({'a': 3, 'b': 1, 'c': 1})
print(counts.most_common(1))     # [('a', 3)]

groups = defaultdict(list)
groups["fruits"].append("apple")   # no KeyError, even though "fruits" didn't exist yet

Point = namedtuple("Point", ["x", "y"])
p1 = Point(3, 4)
print(p1.x, p1.y)                # 3 4 — named AND indexed access both work

dq = deque([1, 2, 3])
dq.appendleft(0)                   # O(1) - a plain list's insert(0, ...) is O(n)
```

15.2 itertools — Iterator Building Blocks

```
import itertools

print(list(itertools.chain([1, 2], [3, 4])))         # [1, 2, 3, 4]
print(list(itertools.permutations([1, 2, 3], 2)))     # all 2-length orderings
print(list(itertools.combinations([1, 2, 3], 2)))      # all 2-length unordered picks
print(list(itertools.islice(itertools.count(10), 3)))    # [10, 11, 12] — a lazy infinite counter, sliced
```

15.3 datetime — Dates and Times

```
from datetime import datetime, timedelta

now = datetime.now()
print(now.strftime("%Y-%m-%d %H:%M:%S"))     # formatted string output

tomorrow = now + timedelta(days=1)             # arithmetic directly on datetime objects
parsed = datetime.strptime("2026-07-10", "%Y-%m-%d")   # parse a string INTO a datetime
```

15.4 os and sys — Operating System Interfaces

```
import os, sys

print(os.getcwd())             # current working directory
print(os.listdir("."))         # list files/folders in a directory
os.makedirs("new_folder", exist_ok=True)

print(sys.argv)                # command-line arguments the script was run with
print(sys.version)             # the running Python interpreter's version string
```

15.5 re — Regular Expressions

```
import re

text = "Contact: ayan@example.com or riya@example.com"
emails = re.findall(r"[\\w.-]+@[\\w.-]+", text)
print(emails)

if re.match(r"^\\d{3}-\\d{4}$", "555-1234"):
    print("Valid phone format")

cleaned = re.sub(r"\\s+", " ", "too    many   spaces")     # collapse whitespace
```

15.6 random — Pseudo-Random Generation

```
import random

print(random.randint(1, 6))        # random int, INCLUSIVE on both ends — like a die roll
print(random.choice(["a", "b", "c"]))
nums = [1, 2, 3, 4, 5]
random.shuffle(nums)               # shuffles the list IN PLACE
```

15.7 Standard Library Quick-Reference

Module	Typical Use
collections	Specialized containers (Counter, defaultdict, deque, namedtuple)
itertools	Efficient, composable iteration tools
datetime	Dates, times, and durations
os / sys / pathlib	Filesystem, environment, and interpreter interaction
re	Regular expression pattern matching
json / csv	Structured data serialization (Section 14.4, 14.5)
math / statistics	Mathematical functions and basic statistics

Module	Typical Use
random	Pseudo-random number generation and sampling
functools	Higher-order function tools (reduce, wraps, lru_cache, partial)
logging	Configurable, leveled application logging (preferred over print for real apps)

16. Modern Python Features

16.1 f-strings (3.6+) — Recap and Expansion

Already introduced in Section 6.1.1, f-strings are the modern standard for string formatting and support arbitrary expressions, method calls, and format specifiers inline.

```
name, score = "Ayan", 92.456
print(f"{name.upper()} scored {score:.1f}%")     # method calls and format specs both work
print(f"{score:>10.2f}")                         # right-aligned, width 10, 2 decimals
```

16.2 Type Hints

Type hint: optional syntax annotating the expected types of variables, parameters, and return values — used by external static type checkers (mypy, pyright) and IDEs for error detection and autocompletion, but NOT enforced by the Python interpreter at runtime

```
def greet(name: str, times: int = 1) -> str:
    return (name + " ") * times

age: int = 22
scores: list[int] = [90, 85, 78]          # generic container type hints (3.9+ syntax)
lookup: dict[str, int] = {"Ayan": 22}
maybe_name: str | None = None             # union type (3.10+ syntax), replacing Optional[str]
```

Note: Because type hints are purely advisory to the interpreter, `def greet(name: str)` will NOT raise an error at runtime if called as `greet(42)` — catching that mismatch requires running a separate static type checker like mypy as part of the development workflow, unlike C++ where the compiler itself enforces types.

16.3 dataclasses (recap and expansion)

```
from dataclasses import dataclass, field
```

```
@dataclass
class Student:
    name: str
    roll: int
    marks: float = 0.0
    tags: list[str] = field(default_factory=list)   # safe way to default to a mutable value

s1 = Student("Ayan", 101, 92.5)
print(s1)                  # Student(name='Ayan', roll=101, marks=92.5, tags=[])
print(s1 == Student("Ayan", 101, 92.5, []))   # True — __eq__ generated automatically
```

Note: `field(default_factory=list)` is the dataclass-safe way to give a field a fresh empty list per instance — it sidesteps the exact mutable-default pitfall described for ordinary functions in Section 5.2, since `tags: list = []` in a dataclass would share one list across every instance, just as it would for a regular function parameter.

16.4 The Walrus Operator := (3.8+)

Walrus operator (:=): an assignment expression that both assigns a value to a name AND evaluates to that value in the same expression, most useful for avoiding a redundant, duplicate call inside conditions and comprehensions

```
# Without the walrus operator:
data = get_data()
if data:
    process(data)

# With the walrus operator — assignment folded directly into the condition:
if (data := get_data()):
    process(data)

# Inside a comprehension, avoiding a duplicate function call:
results = [y for x in values if (y := expensive(x)) is not None]
```

16.5 Structural Pattern Matching — Recap and Expansion

Section 4.2.2 introduced match/case for simple value matching. Pattern matching can also destructure sequences, dicts, and class instances directly in the case clause.

```
def handle(command):
    match command.split():
        case ["go", direction]:
            print(f"Moving {direction}")
        case ["pick", "up", item]:
            print(f"Picking up {item}")
        case ["quit"]:
            print("Goodbye")
        case _:
            print("Unknown command")
```

```
handle("go north")
handle("pick up sword")
```

16.6 Context Managers, Comprehensions, Decorators — Cross-Reference

Several of the most distinctly 'modern-Python' features were already covered where they naturally fit: the with statement / context managers in Section 10.5, comprehensions in Section 12.4, and decorators in Section 13.3. Together with f-strings, type hints, dataclasses, and the walrus operator, these define what is generally meant by idiomatic, current-style Python.

17. Virtual Environments and Package Management

17.1 Why Virtual Environments?

Virtual environment: an isolated Python installation with its own independent set of installed packages, separate from the system-wide Python and from other projects' environments — used to prevent version conflicts between projects that need different (and possibly incompatible) versions of the same package

> **Note:** Installing packages directly into a system-wide Python (especially with `sudo pip install`) can silently break OS tools that depend on specific package versions, and makes it impossible for two projects on the same machine to depend on different versions of the same library. Creating a virtual environment per project is standard practice.

17.2 venv — the Built-in Standard

```
python3 -m venv .venv              # creates a virtual environment in the .venv folder

source .venv/bin/activate          # activate it (Linux/macOS)
.venv\\Scripts\\activate           # activate it (Windows)

pip install requests               # now installs ONLY into this project's .venv
pip freeze > requirements.txt      # capture exact installed versions

deactivate                         # exit the virtual environment
```

17.3 requirements.txt — Reproducing an Environment

```
# requirements.txt
requests==2.31.0
flask>=2.3,<3.0
numpy

# On another machine, recreate the exact same set of packages:
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

17.4 Package Management Quick-Reference

Tool / File	Purpose
pip	Installs packages from PyPI into the currently active Python environment
venv	Built into the standard library; creates lightweight, isolated environments
requirements.txt	A plain-text list of pinned dependencies for reproducible installs
pyproject.toml	The modern, standardized project/dependency metadata file (used by tools like Poetry, uv, Hatch)

Note: Newer tools (Poetry, uv, pipenv) build on top of pip and venv to add dependency locking, resolution, and a single pyproject.toml-based workflow — but venv + pip + requirements.txt remains the baseline, universally-available approach worth knowing first.

18. Best Practices and Common Pitfalls

18.1 General Best Practices

- Follow PEP 8 (Python's official style guide): 4-space indentation, snake_case for variables/functions, PascalCase for classes, and descriptive names.
- Prefer f-strings over .format() or % formatting for new code (Section 6.1.1, 16.1).
- Use list/dict/set comprehensions over manual loops with repeated .append() when the logic stays simple and readable (Section 6.2.2, 12.4).
- Use context managers (with) for anything requiring guaranteed cleanup — files, network connections, locks — instead of relying on __del__ (Section 10.5).
- Never use a mutable default argument (list, dict, set); use None and create the mutable object inside the function body instead (Section 5.2).
- Catch specific exception types rather than a bare except:, and order except clauses from most specific to most general (Section 10.3).
- Use virtual environments per project, and pin dependencies in a requirements.txt or pyproject.toml (Section 17).
- Add type hints to function signatures in non-trivial codebases, and check them with a tool like mypy (Section 16.2).
- Use is only for identity comparisons (especially against None), and == for value comparisons (Section 3.3).

18.2 Common Pitfalls

Pitfall	Why It's a Problem	Fix
Mutable default argument	The same list/dict object is reused and accumulates state across unrelated calls	Use None as the default, and create the mutable object inside the function body
b = a on a list/dict	Creates an ALIAS, not a copy — mutating one affects both names	Use a.copy(), list(a), or copy.deepcopy(a) for an independent copy (Section 6.4)
Using == instead of is for None	Works by coincidence for None but is not idiomatic and can behave unexpectedly for custom __eq__	Always use 'is None' / 'is not None'
Modifying a list while iterating over it	Skips or repeats elements unpredictably as indices shift underneath the loop	Iterate over a copy (for x in list(original):) or build a new list instead
Bare except: clauses	Silently swallows ALL errors, including KeyboardInterrupt and genuine bugs	Catch specific exception types (Section 10.3)
Off-by-one on range() stop value	range(1, 6) stops BEFORE 6, producing 1..5, not 1..6	Remember range's stop bound is always exclusive (Section 4.3.1)
Integer division surprise	"/" always returns a float in Python 3, unlike C/C++'s integer division for two ints	Use "//" explicitly when floor/integer division is intended (Section 2.4)

18.3 Python vs C/C++ — Summary Table

Feature	C / C++	Python
Typing	Static, compile-time checked	Dynamic; optional hints checked only by external tools
Memory management	Manual (C) or new/delete + smart pointers (C++)	Automatic — reference counting plus a cyclic garbage collector
Blocks	Curly braces {}	Indentation
Access control	Compiler-enforced public/private/protected	Convention only (_name, __name); not enforced
Polymorphism	Requires inheritance + virtual functions	Duck typing works without any shared base class
Collections	Hand-built or STL (vector, map, ...)	Built into the core language (list, dict, set, tuple)
Error handling	Return codes (C) or try/throw/catch (C++)	try/except/finally, used pervasively
Iteration laziness	Manual iterator classes/coroutines for laziness	Native generator functions (yield) and generator expressions
Execution model	Compiled to native machine code	Interpreted (bytecode on a virtual machine)

18.4 Where to Go Next

This document covers the core of Python as commonly used in coursework, interviews, and everyday application development. Deeper topics worth exploring afterward include: async/await and the asyncio event loop for concurrent I/O, multithreading and multiprocessing (and the Global Interpreter Lock's effect on each), metaclasses and descriptors, packaging and distributing your own libraries, and popular frameworks built on top of the language (Django/Flask/FastAPI for web, NumPy/pandas for data). The official docs at docs.python.org remain the most reliable reference for exact language and standard-library behaviour across versions.